Lesson 6: Victory Condition

Unreal Engine (optional advanced track)

VIDEO GAME CREATION

This week: a real win condition. Your job is to track how many enemies are left and show "You Win!" the moment the last one is destroyed - completing your game's Level 1 MVP milestone.

Open your own [YourName]Game project from Lesson 5 - don't start a new one. Everything this week builds on the row of enemies you already have.

Tip: no new assets this week - "You Win!" is shown with each engine's own basic text tools, not an image.

Setup

1. Open your [YourName]Game project from Lesson 5.
2. Open BP_Enemy and add an Event Dispatcher (Details panel under My Blueprint > Event Dispatchers, click +) named OnEnemyDestroyed.
3. Create a new Actor Blueprint named BP_GameManager (Blueprint Class > Actor), and drag one instance of it into the level - it just needs to exist once, and it's the one place win-tracking will live.

Level 1 - Minimal (MVP)

The moment the last enemy is destroyed, "You Win!" appears on screen. This completes your game's Level 1 MVP milestone - a complete, playable core loop.

4. In BP_Enemy's Event Graph, inside the successful-Cast branch (right before the two Destroy Actor nodes), add a "Call OnEnemyDestroyed" node.
5. In BP_GameManager, add an Integer variable EnemiesRemaining, default value 5.
6. In the Level Blueprint, drag your placed BP_GameManager instance from the level (or the World Outliner) into the Event Graph to create a reference to it. In the spawn For Loop, right after each Spawn Actor from Class, add a "Bind Event to OnEnemyDestroyed" node targeting that spawned Actor, wired to that BP_GameManager reference's HandleEnemyDestroyed custom event.
7. In BP_GameManager, build HandleEnemyDestroyed: decrease EnemiesRemaining by 1, Branch on EnemiesRemaining <= 0, and on True, add a Print String node with text "You Win!" and "Print to Screen" checked.
8. Click Compile, then Play and destroy every enemy to test - "You Win!" should appear on screen the instant the last one is destroyed.

```
BP_Enemy's Event Graph (inside the successful Cast branch):
Call OnEnemyDestroyed (Broadcast) -> Destroy Actor(Cast result) -> Destroy
Actor(Self)

Level Blueprint:
Event BeginPlay -> (BP_GameManager reference, dragged in from the level)
->
  For Loop -> Spawn Actor from Class(BP_Enemy, ...) ->
    Bind Event to OnEnemyDestroyed(Target = spawned Actor, Event =
BP_GameManager ref's HandleEnemyDestroyed)

BP_GameManager:
Custom Event HandleEnemyDestroyed ->
  EnemiesRemaining = EnemiesRemaining - 1
  Branch(EnemiesRemaining <= 0) -> True: Print String("You Win!", Print to
Screen = true)
```

Alternate Path: C++ (code, optional advanced track)

The same kill-tracking pattern works in C++ instead of Blueprint nodes.

9. Add a DECLARE_DYNAMIC_MULTICAST_DELEGATE to ACppEnemy.h and a UPROPERTY(BlueprintAssignable) instance of it, then Broadcast() it from OnOverlap() right before calling Destroy().
10. In a GameManager actor class's BeginPlay(), bind a UFUNCTION to each spawned enemy's delegate using AddDynamic(), and track EnemiesRemaining as a member variable - the C++ equivalent of BP_GameManager.
11. In the bound handler, decrement EnemiesRemaining and call GEngine->AddOnScreenDebugMessage() to show "You Win!" when it reaches zero - the C++ equivalent of Print String's Print to Screen option.

```
// ACppEnemy.h
DECLARE_DYNAMIC_MULTICAST_DELEGATE(FOnEnemyDestroyed);
UPROPERTY(BlueprintAssignable)
FOnEnemyDestroyed OnEnemyDestroyed;

// Inside OnOverlap(), before Destroy():
OnEnemyDestroyed.Broadcast();

// In your GameManager's handler:
EnemiesRemaining--;
if (EnemiesRemaining <= 0) {
    GEngine->AddOnScreenDebugMessage(-1, 5.f, FColor::Yellow, TEXT("You
Win!"));
}
```

This mirrors the Blueprint path's Event Dispatcher and Print String node - same underlying engine calls, just from code. Level 2/3's logic already lives the same way in both paths.

Level 2 - Complete (Alpha/Beta)

The win text appears exactly once, exactly when the last enemy is destroyed - not early, not late, and not repeatedly.

- Destroy all 5 enemies and confirm "You Win!" appears exactly once, exactly when the last one is destroyed - not one enemy early, and not repeatedly.

- This already works correctly because each spawned enemy's dispatcher is bound exactly once, at spawn time, always to the same BP_GameManager instance, and Call OnEnemyDestroyed only ever broadcasts once per enemy, right at the moment it's actually destroyed.

Level 3 - Refined (Ready to Publish) - pick any that interest you

- Sound: import the provided sfx_win.wav as a Sound Wave asset, then add a Play Sound 2D node right before the Print String node in BP_GameManager's HandleEnemyDestroyed.
- Visual: use Print String's Text Color parameter to make the win message gold or bright green instead of the default white, and increase Duration so it stays on screen longer.
- Structure: keep all counting logic (EnemiesRemaining, HandleEnemyDestroyed) inside BP_GameManager - the Level Blueprint only ever spawns enemies and binds their dispatchers, and BP_Enemy only ever broadcasts that it was destroyed. Neither one knows or cares how many enemies remain overall.

If Something's Wrong

- "You Win!" never appears: confirm Bind Event to OnEnemyDestroyed is actually wired for every spawned enemy inside the For Loop, and that it's targeting BP_GameManager's HandleEnemyDestroyed, not a Custom Event on the Level Blueprint itself.
- "You Win!" appears too early: confirm EnemiesRemaining's default value on BP_GameManager actually matches your For Loop's enemy count.
- Compile error about the event signature: confirm HandleEnemyDestroyed's parameters (or lack of them) match OnEnemyDestroyed's Event Dispatcher signature exactly.

Want More?

dev.epicgames.com/community/learning, Student Resource List > Unreal